SIMATS ENGINEERING
ASSESSMENT TOOL 3
CO2 – AT3 - Dry Run Evaluation (Solutions with Program Code)

Question 1: Greedy Best-First Search Data.

Undirected road network with step costs and heuristic values h(n) (estimated cost to goal G):

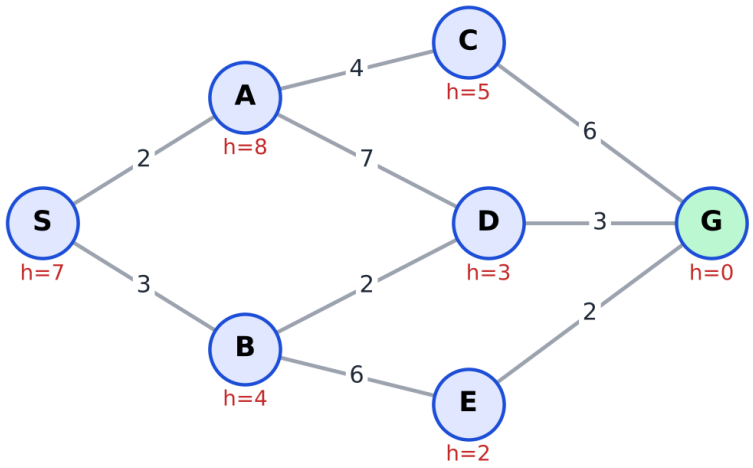


Figure 1: Road network with step costs and heuristic values h(n)

Method. Greedy best-first search keeps an open list ordered only by h(n). At each step it expands the node with the smallest h(n), adds its unvisited neighbours (with their h values) to the open list, and stops when G is expanded. Cost g(n) is ignored.

(a) Dry run table

Initial open list: S:7.

Step	Node expanded	Nodes added to open list	Open list after expansion (node: h)
1	S (h = 7)	A:8, B:4	B:4, A:8
2	B (h = 4)	D:3, E:2	E:2, D:3, A:8
3	E (h = 2)	G:0	G:0, D:3, A:8
4	G (h = 0) – GOAL	–	D:3, A:8

Order of expansion: S, B, E, G. Closed list at the end: {S, B, E, G}.

(b) Path found and total cost

Path: S → B → E → G Cost = 3 + 6 + 2 = 11.

(c) Is the path optimal? (2 marks)

No. Consider the alternative path S → B → D → G, whose cost is 3 + 2 + 3 = **8**, which is less than 11. The other two paths, S → A → C → G (2 + 4 + 6 = 12) and S → A → D → G (2 + 7 + 3 = 12), are more expensive, so the optimal cost is 8.

Reason: at B the algorithm chose E (h = 2) over D (h = 3) because it looks only at the estimated remaining cost and ignores the cost already paid. The edge B–E costs 6 whereas B–D costs only 2,

so the apparently “closer” node led to a more expensive route. Greedy search is therefore fast but neither optimal nor guaranteed to find the cheapest path.

Program :

```
"""
Q1 Greedy Best-First Search
"""
import heapq
from math import inf

# ---------------------------------------------------------------
# Road network (undirected) and heuristic values h(n)
# ---------------------------------------------------------------
EDGES = [("S", "A", 2), ("S", "B", 3), ("A", "C", 4), ("A", "D", 7),
         ("B", "D", 2), ("B", "E", 6), ("C", "G", 6), ("D", "G", 3),
         ("E", "G", 2)]
H = {"S": 7, "A": 8, "B": 4, "C": 5, "D": 3, "E": 2, "G": 0}

GRAPH = {n: {} for n in H}
for u, v, c in EDGES:
    GRAPH[u][v] = c
    GRAPH[v][u] = c

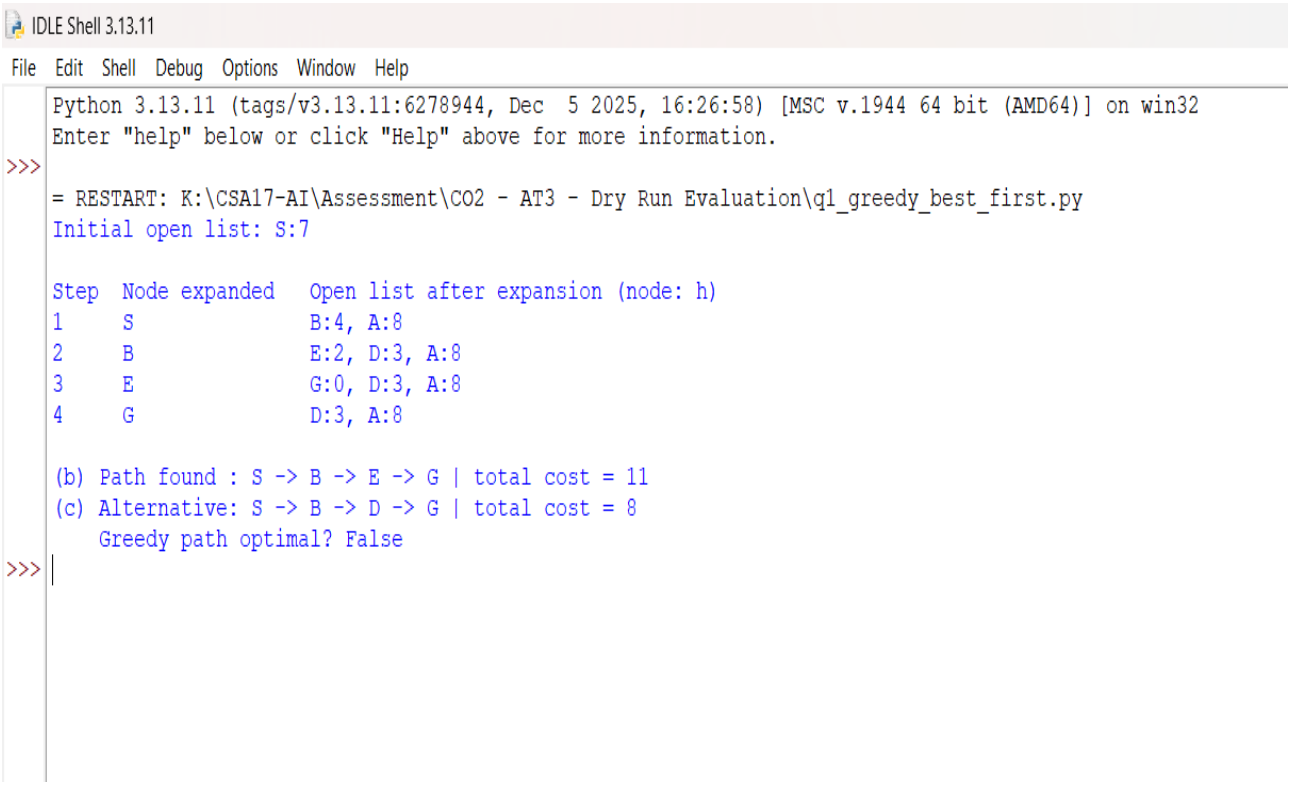

def path_cost(path):
    return sum(GRAPH[a][b] for a, b in zip(path, path[1:]))


def build_path(parent, goal):
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = parent[node]
    return path[::-1]


# ---------------------------------------------------------------
# Q1 - Greedy Best-First Search : priority = h(n)
# ---------------------------------------------------------------
def greedy_best_first(start, goal):
    open_list = {start: H[start]}
    parent = {start: None}
    closed = set()
    steps = []
    while open_list:
        node = min(open_list, key=lambda n: (open_list[n], n))
        del open_list[node]
        closed.add(node)
        if node == goal:
            steps.append((node, dict(open_list)))
            break
        for nb in GRAPH[node]:
            if nb not in closed and nb not in open_list:
                open_list[nb] = H[nb]
```

```
            parent[nb] = node
        steps.append((node, dict(open_list)))
    return build_path(parent, goal), steps
```

Output :



Question 2: A* Search Method.

A* orders the open list by f(n) = g(n) + h(n), where g(n) is the cheapest known cost from S to n. If a cheaper route to a node is found, its g and parent are updated. The search stops when G is removed from the open list.

(a) Dry run table

Initial open list: S: 0 + 7 = 7.

Step	Node expanded (g + h = f)	Open list after expansion (node: g + h = f)
1	S (0 + 7 = 7)	B: 3 + 4 = 7, A: 2 + 8 = 10
2	B (3 + 4 = 7)	D: 5 + 3 = 8, A: 2 + 8 = 10, E: 9 + 2 = 11
3	D (5 + 3 = 8)	G: 8 + 0 = 8, A: 2 + 8 = 10, E: 9 + 2 = 11
4	G (8 + 0 = 8) – GOAL	A: 2 + 8 = 10, E: 9 + 2 = 11

Notes: in step 2, g(D) = g(B) + 2 = 5 and g(E) = g(B) + 6 = 9. In step 3, going from D to A would give g(A) = 5 + 7 = 12, which is worse than the existing g(A) = 2, so A is not updated; G gets g(G) = 5 + 3 = 8. Order of expansion: S, B, D, G.

(b) Path, cost and comparison with Question 1

Path: S → B → D → G Cost = 3 + 2 + 3 = 8.

	Greedy (Q1)	A* (Q2)
Evaluation function	h(n)	g(n) + h(n)
Nodes expanded	S, B, E, G (4)	S, B, D, G (4)
Path found	S → B → E → G	S → B → D → G
Path cost	11	8
Optimal?	No	Yes

A* expanded the same number of nodes as greedy search here but returned a path that is 3 units cheaper, because after B it compared f(D) = 8 with f(E) = 11 and correctly preferred D. Greedy search only compared h(D) = 3 with h(E) = 2.

(c) Admissibility of h(n)

A heuristic is admissible if h(n) ≤ h*(n) for every node, where h*(n) is the true cheapest remaining cost to G. The true costs below were obtained by inspecting the cheapest paths (and confirmed with Dijkstra's algorithm in the program):

Node	Cheapest path to G	True cost h*(n)	h(n)	h(n) ≤ h*(n)?
S	S → B → D → G = 3 + 2 + 3	8	7	Yes (7 ≤ 8)
A	A → D → G = 7 + 3 (or A → C → G = 4 + 6)	10	8	Yes (8 ≤ 10)
B	B → D → G = 2 + 3	5	4	Yes (4 ≤ 5)
C	C → G = 6	6	5	Yes (5 ≤ 6)

Node	Cheapest path to G	True cost h*(n)	h(n)	h(n) ≤ h*(n)?
D	D → G = 3	3	3	Yes (3 ≤ 3)
E	E → G = 2	2	2	Yes (2 ≤ 2)

Conclusion: h(n) never overestimates, so it is admissible (all six non-goal nodes were checked, and h(G) = 0). It is also consistent: h(u) ≤ c(u, v) + h(v) holds for every edge.
Why it matters: A* with an admissible heuristic is guaranteed to return an optimal solution, because f(n) never overestimates the cost of the best path through n, so a cheaper path can never be hidden behind a node with a larger f value. If h overestimated (for example h(D) = 9), A* could ignore the good route S → B → D → G and return a costlier one. With a consistent heuristic, A* also never needs to re-expand a closed node.

Program 1: Greedy Best-First Search and A* (Questions 1 and 2) – q1_q2_search.py

```
# ---------------------------------------------------------------
# Q2 - A* Search : priority = f(n) = g(n) + h(n)
# ---------------------------------------------------------------
def a_star(start, goal):
    g = {start: 0}
    parent = {start: None}
    open_list = {start: H[start]}          # node -> f
    closed = set()
    steps = []
    while open_list:
        node = min(open_list, key=lambda n: (open_list[n], n))
        del open_list[node]
        if node == goal:
            steps.append((node, {n: (g[n], H[n], g[n] + H[n]) for n in open_list}))
            break
        closed.add(node)
        for nb, cost in GRAPH[node].items():
            new_g = g[node] + cost
            if new_g < g.get(nb, inf):
                g[nb] = new_g
                parent[nb] = node
                open_list[nb] = new_g + H[nb]
                closed.discard(nb)
        steps.append((node, {n: (g[n], H[n], g[n] + H[n]) for n in open_list}))
    return build_path(parent, goal), g[goal], steps
# ---------------------------------------------------------------
# Admissibility check : true remaining cost h*(n) by Dijkstra from G
# ---------------------------------------------------------------
def true_costs(goal):
    dist = {n: inf for n in GRAPH}
    dist[goal] = 0
    pq = [(0, goal)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, c in GRAPH[u].items():
```

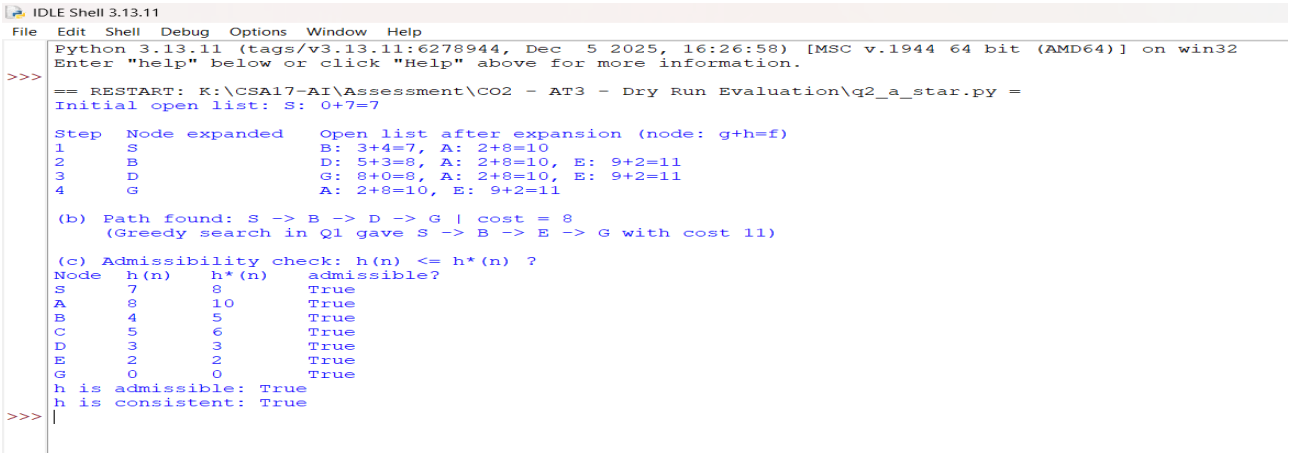
```
            if d + c < dist[v]:
                dist[v] = d + c
                heapq.heappush(pq, (dist[v], v))
    return dist
def show_open_greedy(open_dict):
    items = sorted(open_dict.items(), key=lambda kv: (kv[1], kv[0]))
    return ", ".join(f"{n}:{h}" for n, h in items) or "(empty)"
def show_open_astar(open_dict):
    items = sorted(open_dict.items(), key=lambda kv: (kv[1][2], kv[0]))
    return ", ".join(f"{n}:{g}+{h}={f}" for n, (g, h, f) in items) or "(empty)"


if __name__ == "__main__":
    print("=== Q1 : GREEDY BEST-FIRST SEARCH ===")
    path, steps = greedy_best_first("S", "G")
    for i, (node, op) in enumerate(steps, 1):
        print(f"Step {i}: expand {node:<2} open = {show_open_greedy(op)}")
    print("Path :", " -> ".join(path), "| cost =", path_cost(path))
    alt = ["S", "B", "D", "G"]
    print("Alternative path:", " -> ".join(alt), "| cost =", path_cost(alt))

    print("\n=== Q2 : A* SEARCH ===")
    path, cost, steps = a_star("S", "G")
    for i, (node, op) in enumerate(steps, 1):
        print(f"Step {i}: expand {node:<2} open = {show_open_astar(op)}")
    print("Path :", " -> ".join(path), "| cost =", cost)

    print("\n=== Admissibility check ===")
    true = true_costs("G")
    print(f"{'Node':<5}{'h(n)':<6}{'h*(n)':<7}admissible?")
    for n in "SABCDEG":
        print(f"{n:<5}{H[n]:<6}{true[n]:<7}{H[n] <= true[n]}")
    print("All admissible:", all(H[n] <= true[n] for n in H))
    consistent = all(H[u] <= c + H[v] for u in GRAPH for v, c in GRAPH[u].items())
    print("Consistent    :", consistent)
```

Output :



Question 3: Constraint Satisfaction Problem

(a) CSP formulation and constraint graph

- **Variables:** WA, NT, SA, Q, NSW, V (the six regions).
- **Domains:** D(WA) = D(NT) = D(SA) = D(Q) = D(NSW) = D(V) = {R, G, B}.
- **Constraints (binary, ≠):** WA ≠ NT, WA ≠ SA, NT ≠ SA, NT ≠ Q, SA ≠ Q, SA ≠ NSW, SA ≠ V, Q ≠ NSW, NSW ≠ V (9 constraints – one per pair of neighbouring regions).

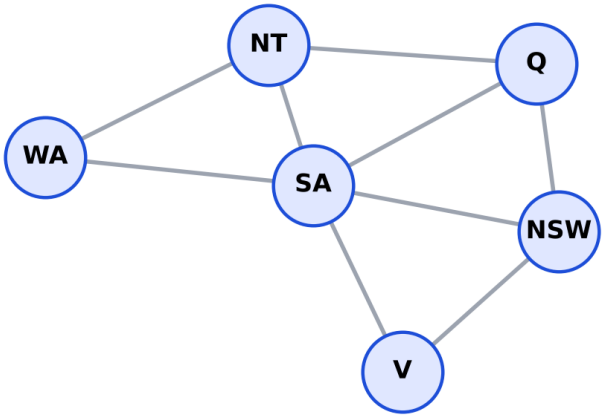


Figure 2: Constraint graph (nodes = regions, edges = "must differ in colour")

Degrees: SA = 5, NT = 3, Q = 3, NSW = 3, WA = 2, V = 2 (sum 18 = 2 × 9 edges).

(b) Backtracking with forward checking, MRV and degree heuristic

Procedure: select the unassigned variable with the fewest legal values (MRV); break ties by the largest number of unassigned neighbours (degree); then by the order WA, NT, SA, Q, NSW, V. Try colours in the order R, G, B. After each assignment, forward checking removes that colour from the domains of unassigned neighbours; an empty domain would force backtracking.

Initial domains: WA {R,G,B}, NT {R,G,B}, SA {R,G,B}, Q {R,G,B}, NSW {R,G,B}, V {R,G,B}.

Step	Variable chosen (reason)	Assign	Forward checking removes	Domains of unassigned regions afterwards
1	All domains size 3 → highest degree: SA (5)	SA = R	R from WA, NT, Q, NSW, V	WA{G,B} NT{G,B} Q{G,B} NSW{G,B} V{G,B}
2	MRV tie (all 2); degree NT = Q = NSW = 2 → order picks NT	NT = G	G from WA, Q	WA{B} Q{B} NSW{G,B} V{G,B}
3	MRV tie WA{B}, Q{B}; degree WA = 0, Q = 1 → Q	Q = B	B from NSW	WA{B} NSW{G} V{G,B}
4	MRV tie WA{B}, NSW{G}; degree WA = 0, NSW = 1 → NSW	NSW = G	G from V	WA{B} V{B}

Step	Variable chosen (reason)	Assign	Forward checking removes	Domains of unassigned regions afterwards
5	MRV tie WA{B}, V{B}; degree 0 = 0 → order picks WA	WA = B	nothing (no unassigned neighbours)	V{B}
6	Only V left	V = B	nothing	– (all assigned)

Final colouring: SA = Red, NT = Green, Q = Blue, NSW = Green, WA = Blue, V = Blue. Check: WA–NT (B,G), WA–SA (B,R), NT–SA (G,R), NT–Q (G,B), SA–Q (R,B), SA–NSW (R,G), SA–V (R,B), Q–NSW (B,G), NSW–V (G,B) – all different. The search needed 6 assignments and no backtracking.

(c) Effect of the MRV and degree heuristics

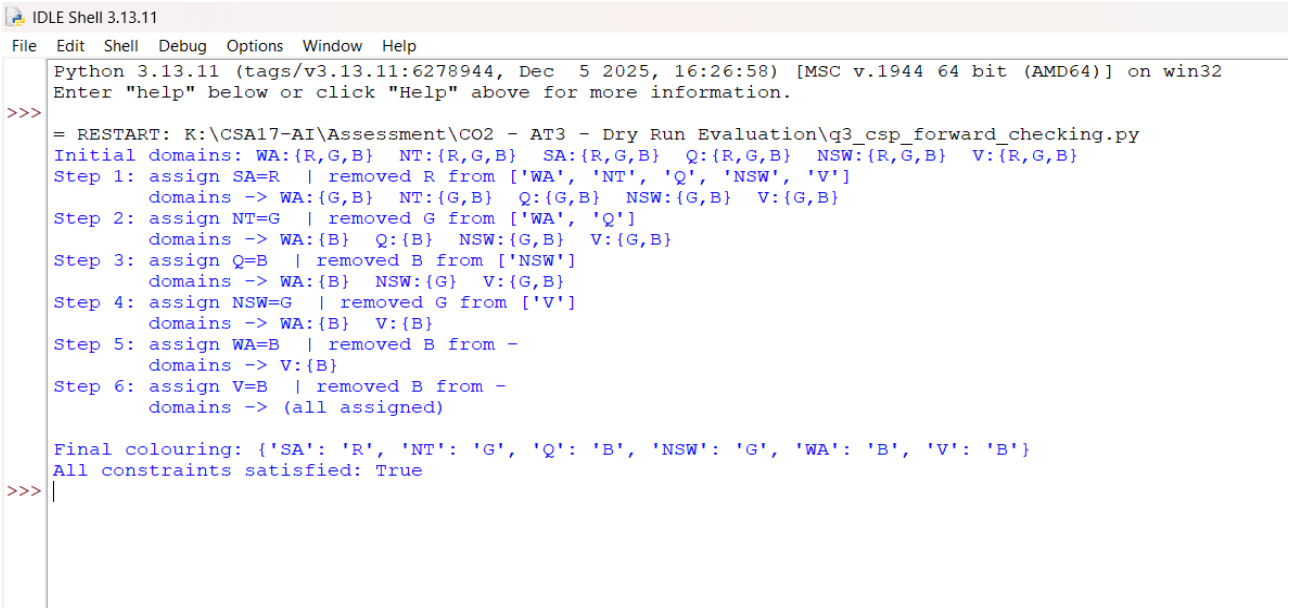
- **MRV (fail-first):** choosing the variable with the fewest remaining values means the most constrained region is coloured first, so conflicts show up early and wrong branches are cut near the root instead of deep in the tree. In the run above, after SA = R and NT = G, the regions WA and Q were already down to a single value, so they were assigned next and their colours were forced – no branching at all.
- **Degree heuristic (tie-breaker):** at the start every region has three colours, so MRV cannot decide. Choosing SA, which touches five other regions, lets forward checking prune the most values in one move (it reduced every other domain from 3 to 2). Later, picking Q (degree 1) before WA (degree 0) also leaves fewer constraints on the remaining variables.
- Together they reduce the effective branching factor and the number of nodes explored; here the whole solution was found with the minimum possible 6 assignments and zero backtracks.

Program : Backtracking + Forward Checking + MRV + Degree (Question 3) – q3_csp.py

```
"""
Q3 - CSP (map colouring) : backtracking + forward checking + MRV + degree heuristic
"""
VARS = ["WA", "NT", "SA", "Q", "NSW", "V"]
DOMAIN = ["R", "G", "B"]
EDGES = [("WA", "NT"), ("WA", "SA"), ("NT", "SA"), ("NT", "Q"), ("SA", "Q"),
        ("SA", "NSW"), ("SA", "V"), ("Q", "NSW"), ("NSW", "V")]
NB = {v: set() for v in VARS}
for a, b in EDGES:
    NB[a].add(b)
    NB[b].add(a)
def fmt(domains, assigned):
    return "  ".join(f"{v}:{{{','.join(domains[v])}}}" for v in VARS if v not in assigned)
def select_variable(domains, assigned):
    unassigned = [v for v in VARS if v not in assigned]
    # MRV -> degree (most unassigned neighbours) -> fixed order
    return min(unassigned,
               key=lambda v: (len(domains[v]),
                              -len([n for n in NB[v] if n not in assigned]),
                              VARS.index(v)))
def backtrack(assigned, domains, log, depth=0):
    if len(assigned) == len(VARS):
```

```
        return assigned
    var = select_variable(domains, assigned)
    for colour in DOMAIN:
        if colour not in domains[var]:
            continue
        new_assigned = dict(assigned, **{var: colour})
        new_domains = {v: list(d) for v, d in domains.items()}
        new_domains[var] = [colour]
        pruned, failed = [], False
        for n in NB[var]:                    # forward checking
            if n not in new_assigned and colour in new_domains[n]:
                new_domains[n].remove(colour)
                pruned.append(n)
                if not new_domains[n]:
                    failed = True
        log.append((depth + 1, var, colour, fmt(new_domains, new_assigned),
                    pruned, "DEAD END - backtrack" if failed else ""))
        if not failed:
            result = backtrack(new_assigned, new_domains, log, depth + 1)
            if result:
                return result
    return None
if __name__ == "__main__":
    domains = {v: list(DOMAIN) for v in VARS}
    log = []
    solution = backtrack({}, domains, log)
    print("Initial domains:", fmt(domains, {}))
    for i, (d, var, col, doms, pruned, note) in enumerate(log, 1):
        print(f"Step {i}: assign {var}={col}  | removed {col} from {pruned or '-'}")
        print("        domains ->", doms or "(all assigned)", note)
    print("\nFinal colouring:", solution)
    ok = all(solution[a] != solution[b] for a, b in EDGES)
    print("All constraints satisfied:", ok)
```

Output :



Question 4: Minimax with a Heuristic Evaluation Function

(a) Minimax backed-up values

Naming: A is the root (MAX); B1, B2 are the MIN children; C1–C4 are the MAX grandchildren; the eight leaves are, left to right, 3, 5 | 6, 9 | 1, 2 | 0, −1. MAX nodes take the maximum of their children, MIN nodes the minimum.

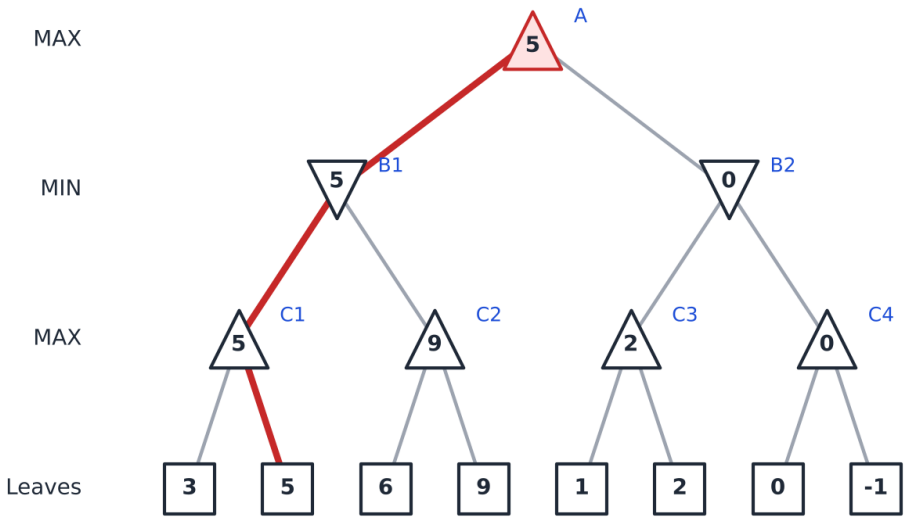


Figure 3: Game tree with backed-up minimax values (red = principal variation)

Node	Type	Children values	Backed-up value
C1	MAX	max(3, 5)	5
C2	MAX	max(6, 9)	9
C3	MAX	max(1, 2)	2
C4	MAX	max(0, −1)	0
B1	MIN	min(C1, C2) = min(5, 9)	5
B2	MIN	min(C3, C4) = min(2, 0)	0
A (root)	**MAX**	**max(B1, B2) = max(5, 0)**	**5**

(b) Best first move and principal variation

Best first move: Left (to B1), because it is worth 5 against 0 for the right move. The minimax value of the game is 5.

Principal variation: A (MAX) → B1 (MIN) → C1 (MAX) → leaf 5. That is, MAX moves left, MIN then chooses C1 (value 5, smaller than C2's 9), and MAX finally takes the right leaf of C1 (5 is larger than 3).

(c) Heuristic evaluation of the tic-tac-toe board

E = (number of lines still open for X) − (number of lines still open for O). A line is open for X if it contains no O, and open for O if it contains no X In the board shown, X occupies the top-left and centre squares and O occupies the top-middle and bottom-right squares.

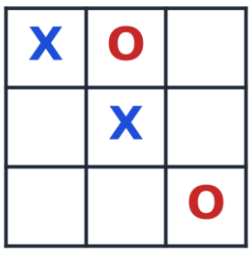


Figure 4: The board

Line	Cells	Open for X?	Open for O?
Row 1 (top)	X, O, –	No (has O)	No (has X)
Row 2 (middle)	–, X, –	Yes	No (has X)
Row 3 (bottom)	–, –, O	No (has O)	Yes
Column 1	X, –, –	Yes	No (has X)
Column 2	O, X, –	No (has O)	No (has X)
Column 3	–, –, O	No (has O)	Yes
Diagonal (top-left to bottom-right)	X, X, O	No (has O)	No (has X)
Diagonal (top-right to bottom-left)	–, X, –	Yes	No (has X)
Total		**3**	**2**

E = 3 − 2 = +1 (slightly favourable for X).

Program 4 :

```
"""
Q4 - Minimax on the given game tree + tic-tac-toe evaluation function E
"""

# ====================== Q4 (a), (b) : Minimax ======================
LEAVES4 = [3, 5, 6, 9, 1, 2, 0, -1]

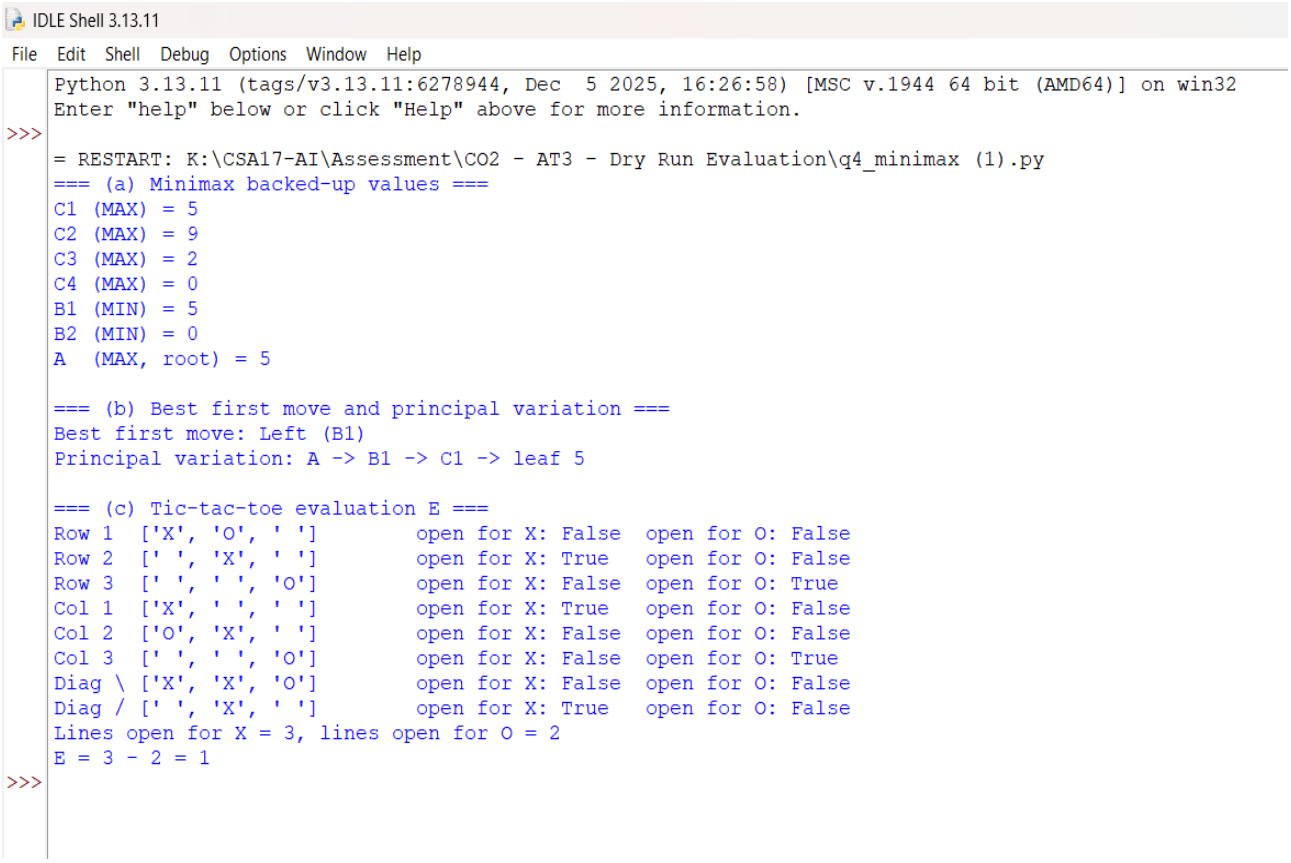

def minimax_q4(leaves):
    max_nodes = [max(leaves[i], leaves[i + 1]) for i in range(0, 8, 2)]   # C1..C4
    min_nodes = [min(max_nodes[0], max_nodes[1]),
                 min(max_nodes[2], max_nodes[3])]                         # B1, B2
    root = max(min_nodes)                                                 # A
    return max_nodes, min_nodes, root


# ============ Q4 (c) : E = (lines open for X) - (lines open for O) ============
LINES = [[(0, 0), (0, 1), (0, 2)], [(1, 0), (1, 1), (1, 2)], [(2, 0), (2, 1), (2, 2)],
         [(0, 0), (1, 0), (2, 0)], [(0, 1), (1, 1), (2, 1)], [(0, 2), (1, 2), (2, 2)],
         [(0, 0), (1, 1), (2, 2)], [(0, 2), (1, 1), (2, 0)]]
NAMES = ["Row 1", "Row 2", "Row 3", "Col 1", "Col 2", "Col 3", "Diag \\", "Diag /"]
```

```
BOARD = [["X", "O", " "],
         [" ", "X", " "],
         [" ", " ", "O"]]


def evaluate(board):
    open_x = open_o = 0
    detail = []
    for name, line in zip(NAMES, LINES):
        cells = [board[r][c] for r, c in line]
        x_open = "O" not in cells       # no O in the line -> still open for X
        o_open = "X" not in cells       # no X in the line -> still open for O
        open_x += x_open
        open_o += o_open
        detail.append((name, cells, x_open, o_open))
    return open_x, open_o, detail
```

Output :



Question 5: Alpha-Beta Pruning

Tree: depth 4, binary. Levels: root A (MAX) → B1, B2 (MIN) → C1–C4 (MAX) → D1–D8 (MIN) → 16 leaves. Leaves left to right: 8, 6 | 5, 7 | 2, 9 | 4, 3 | 6, 1 | 5, 2 | 9, 7 | 3, 4 (each pair is under one of D1–D8). Children are visited left to right, starting with (α, β) = (−∞, +∞). A branch is cut when α ≥ β.

(a) Alpha-beta trace, pruned branches and root value (6 marks)

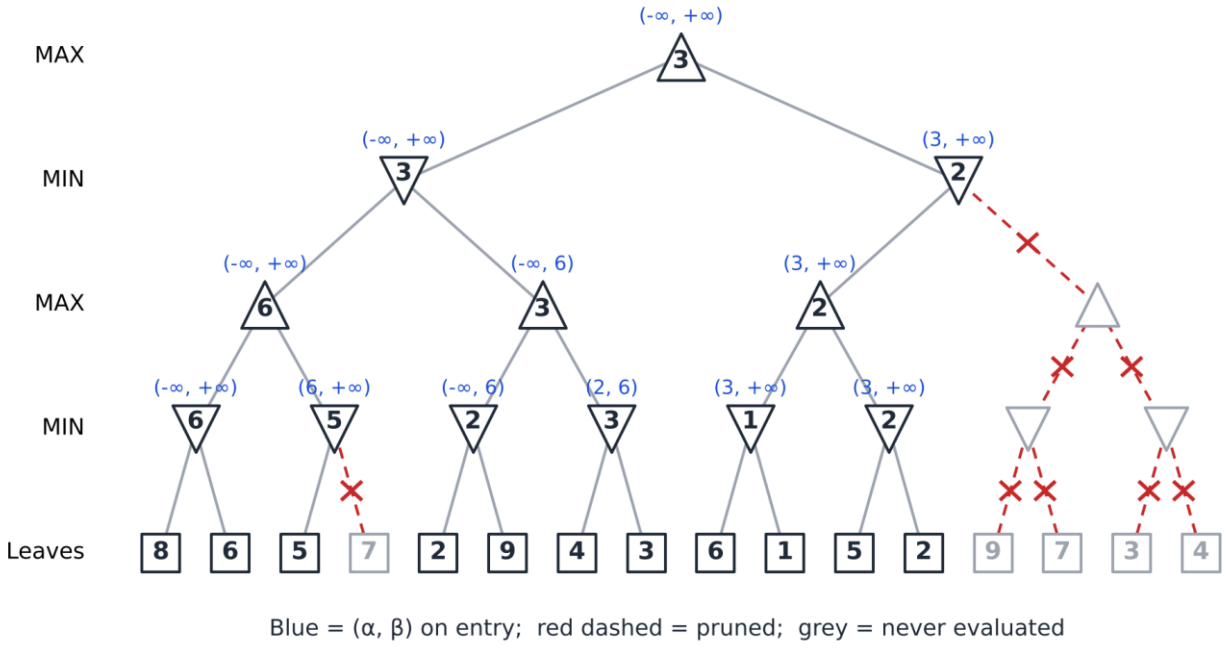


Figure 5: Alpha-beta search tree (blue = (α, β) on entry, red dashed = pruned)

Node	Type	(α, β) on entry	What happens	Returns
A	**MAX**	**(−∞, +∞)**	**Visits B1, then B2; final α = 3**	**3**
B1	MIN	(−∞, +∞)	C1 = 6 gives β = 6; C2 = 3 gives β = 3	3
C1	MAX	(−∞, +∞)	D1 = 6 gives α = 6; D2 = 5 (no change)	6
D1	MIN	(−∞, +∞)	leaf 8 → β = 8; leaf 6 → β = 6	6
D2	MIN	(6, +∞)	leaf 5 → β = 5 ≤ α = 6: CUT-OFF, leaf 7 PRUNED	5
C2	MAX	(−∞, 6)	D3 = 2 gives α = 2; D4 = 3 gives α = 3	3
D3	MIN	(−∞, 6)	leaf 2 → β = 2; leaf 9 (no change)	2
D4	MIN	(2, 6)	leaf 4 → β = 4; leaf 3 → β = 3 (> α = 2, no cut)	3
B2	MIN	(3, +∞)	C3 = 2 gives β = 2 ≤ α = 3: CUT-OFF, C4 PRUNED	2
C3	MAX	(3, +∞)	D5 = 1, D6 = 2; α stays 3	2
D5	MIN	(3, +∞)	leaf 6 → β = 6; leaf 1 → β = 1 (≤ α, but no sibling left to prune)	1

Node	Type	(α, β) on entry	What happens	Returns
D6	MIN	(3, +∞)	leaf 5 → β = 5; leaf 2 → β = 2 (≤ α, no sibling left)	2
C4, D7, D8	–	not visited	Pruned: leaves 9, 7, 3, 4 never evaluated	–

Pruned branches: (1) the right leaf (7) under D2, because β = 5 ≤ α = 6; (2) the whole subtree C4 (D7 and D8, leaves 9, 7, 3, 4) under B2, because B2's β = 2 ≤ α = 3 from the root.
Value of the root: 3 (A = max(B1, B2) = max(3, 2)). MAX's best move is the left branch to B1.

(b) Number of leaves evaluated and comparison with minimax (2 marks)

	Plain minimax	Alpha-beta
Leaves evaluated	16	11 (8, 6, 5, 2, 9, 4, 3, 6, 1, 5, 2)
Leaves skipped	0	5 (7, 9, 7, 3, 4) – about 31% fewer
Root value	3	3

Plain minimax gives D-level values 6, 5, 2, 3, 1, 2, 7, 3, then C values 6, 3, 2, 7, then B1 = min(6, 3) = 3, B2 = min(2, 7) = 2 and root = max(3, 2) = 3. Alpha-beta returns the same root value, 3, as it must: pruning only removes branches that cannot affect the final decision.

(c) Effect of move ordering (2 marks)

Alpha-beta pruning removes more of the tree when the best moves are examined first, because tight bounds (α for MAX, β for MIN) are established early and later siblings can be cut quickly. With perfect ordering the number of nodes examined is about O(b^(d/2)), so the search can go roughly twice as deep in the same time; with the worst ordering nothing is pruned and the cost is O(b^d), the same as minimax.

Example from this tree: D2 has leaves (5, 7) under α = 6. Because 5 is looked at first, β = 5 ≤ 6 immediately and leaf 7 is skipped. If the order were (7, 5), the 7 would have to be evaluated first (β = 7 > 6) and both leaves would be examined, so no pruning would occur. In practice good ordering is obtained with heuristics such as trying captures or previous best moves first, or by iterative deepening.

Program 5: tic-tac-toe evaluation and Alpha-Beta (Questions 5) – q5_games.py

```
# ====================== Q5 : Alpha-Beta pruning ======================
LEAVES5 = [8, 6, 5, 7, 2, 9, 4, 3, 6, 1, 5, 2, 9, 7, 3, 4]
INF = float("inf")


def plain_minimax(depth, index, is_max, leaves, counter):
    if depth == 4:
        counter[0] += 1
        return leaves[index]
    vals = [plain_minimax(depth + 1, 2 * index + i, not is_max, leaves, counter)
            for i in (0, 1)]
    return max(vals) if is_max else min(vals)
```

```
def alpha_beta(depth, index, is_max, alpha, beta, leaves, stats, name):
    if depth == 4:                              # leaf
        stats["leaves"] += 1
        stats["trace"].append(f"{name}: leaf = {leaves[index]}")
        return leaves[index]
    stats["trace"].append(f"{name} ({'MAX' if is_max else 'MIN'}) "
                          f"entry alpha={alpha} beta={beta}")
    value = -INF if is_max else INF
    for i in (0, 1):
        v = alpha_beta(depth + 1, 2 * index + i, not is_max,
                       alpha, beta, leaves, stats, f"{name}{i}")
        if is_max:
            value = max(value, v)
            alpha = max(alpha, value)
        else:
            value = min(value, v)
            beta = min(beta, value)
        if alpha >= beta:                       # cut-off
            if i == 0:
                stats["pruned"].append(
                    f"{name}1 pruned (alpha={alpha} >= beta={beta})")
            break
    stats["trace"].append(f"{name} returns {value}")
    return value


if __name__ == "__main__":
    print("=== Q4(a),(b) Minimax ===")
    mx, mn, root = minimax_q4(LEAVES4)
    print("MAX nodes C1..C4:", mx)
    print("MIN nodes B1,B2 :", mn)
    print("Root A          :", root)
    print("Best first move :", "Left (B1)" if mn[0] >= mn[1] else "Right (B2)")

    print("\n=== Q4(c) Tic-tac-toe evaluation ===")
    ox, oo, detail = evaluate(BOARD)
    for name, cells, xo, oo_ in detail:
        print(f"{name:<7}{str(cells):<22} open for X: {xo!s:<6} open for O: {oo_}")
    print(f"Lines open for X = {ox}, for O = {oo}, E = {ox - oo}")

    print("\n=== Q5 Alpha-Beta ===")
    stats = {"leaves": 0, "trace": [], "pruned": []}
    root5 = alpha_beta(0, 0, True, -INF, INF, LEAVES5, stats, "n")
    print("\n".join(stats["trace"]))
    print("\nPruned branches:", *stats["pruned"], sep="\n  ")
    counter = [0]
    root_plain = plain_minimax(0, 0, True, LEAVES5, counter)
    print(f"\nAlpha-beta : root = {root5}, leaves evaluated = {stats['leaves']}")
    print(f"Minimax    : root = {root_plain}, leaves evaluated = {counter[0]}")
    print("Same root value:", root5 == root_plain)
```

Output :


IDLE Shell 3.13.11
File Edit Shell Debug Options Window Help
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: K:\CSA17-AI\Assessment\CO2 - AT3 - Dry Run Evaluation\q5_alpha_beta.py
=== (a) Alpha-beta trace (n = root; digits = child path, 0 left, 1 right) ===
n (MAX) entry alpha=-inf beta=inf
n0 (MIN) entry alpha=-inf beta=inf
n00 (MAX) entry alpha=-inf beta=inf
n000 (MIN) entry alpha=-inf beta=inf
n0000: leaf = 8
n0001: leaf = 6
n000 returns 6
n001 (MIN) entry alpha=6 beta=inf
n0010: leaf = 5
n001 returns 5
n00 returns 6
n01 (MAX) entry alpha=-inf beta=6
n010 (MIN) entry alpha=-inf beta=6
n0100: leaf = 2
n0101: leaf = 9
n010 returns 2
n011 (MIN) entry alpha=2 beta=6
n0110: leaf = 4
n0111: leaf = 3
n011 returns 3
n01 returns 3
n0 returns 3
n1 (MIN) entry alpha=3 beta=inf
n10 (MAX) entry alpha=3 beta=inf
n100 (MIN) entry alpha=3 beta=inf
n1000: leaf = 6
n1001: leaf = 1
n100 returns 1
n101 (MIN) entry alpha=3 beta=inf
n1010: leaf = 5
n1011: leaf = 2
n101 returns 2
n10 returns 2
n1 returns 2
n returns 3
Pruned branches:
n0011 pruned (alpha=6 >= beta=5)
n11 pruned (alpha=3 >= beta=2)
Value of the root: 3
=== (b) Comparison with plain minimax ===
Alpha-beta : root = 3, leaves evaluated = 11 of 16
Minimax : root = 3, leaves evaluated = 16 of 16
Same root value: True
>>>